

Raport ze wstępnych badań projektu zespołowego Big Data

Propozycja trasy na podstawie otwartych danych nawigacyjnych dla miasta stołecznego Warszawy

Zespół 12:

Michał Zalewski

Jakub Próchnicki

Jan Machowski

Maciej Kurzydłowski

Jan Domański

Warszawa, 21 maja 2026

Spis treści

1	Wstęp	3
2	Infrastruktura oraz potok przetwarzania danych	3
2.1	Przedstawienie infrastruktury	3
2.2	Potok danych	4
2.2.1	Źródła danych	4
2.2.2	Pobór danych - Kafka i skrypty ingestujące	5
2.2.3	Czyszczenie i transformacja danych - Apache Spark, OSMnx, Pandas	5
2.2.4	Zapis danych	5
2.2.5	Wykorzystanie danych w analizie tras	5
3	Instalacja, konfiguracja oraz wykonanie projektu	5
3.1	Instalacja i zbudowanie projektu	5
3.2	Konfiguracja skryptów analizy	5
3.3	Dane wyjściowe	6
4	Wstępna analiza problemu	6
4.1	Wnioski	6
5	Dalsze kierunki badania	6
6	Podsumowanie	6

1 Wstęp

Niniejszy raport jest kontynuacją prac dążących do opracowania modelu propozycji trasy na podstawie otwartych danych nawigacyjnych. Główny cel tego dokumentu to przedstawienie stworzonej w ramach projektu infrastruktury oraz potoku przetwarzania danych. Opisana zostanie również wstępna analiza wyżej wymienionego problemu oraz wnioski wynikające z tej analizy.

W poprzednim dokumencie zawarty został przegląd tematyki, wstępna propozycja źródeł oraz narzędzi wykorzystanych do wykonania projektu, oraz problemów i ograniczeń związanych z wybranymi danymi.

Dla przypomnienia - zaproponowana została reprezentacja miasta w postaci grafu skierowanego, gdzie wierzchołki są przecięciami lub zakończeniami dróg a krawędzie są faktycznymi odcinkami dróg, z odpowiednim zestawem atrybutów - długość, typ drogi, liczba pasów, natężenie ruchu etc. Propozycja ta została wykorzystana do stworzenia wstępnego rozwiązania, które zostanie przedstawione poniżej.

2 Infrastruktura oraz potok przetwarzania danych

2.1 Przedstawienie infrastruktury

Projekt został przygotowany jako środowisko kontenerowe - uruchamiany jest za pomocą **Docker Compose**. Pozwala to na odtworzenie środowiska na różnych urządzeniach, nie musząc instalować ręcznie zależności systemowych / bibliotek. W kontenerze **opennavigation** znajdują się narzędzia potrzebne do pobierania, przetwarzania i analizy danych.

Język **Python** został wybrany jako główny język implementacji. Jednakże **OSMnx** i **NetworkX** są wykorzystywane do pracy z grafem drogowym Warszawy.

Pandas służy w projekcie do przetwarzania danych tabelarycznych, agregacji wyników, zapisu plików wynikowych.

W ramach konteneru zainstalowane są także narzędzia **Apache Kafka** oraz **Apache Spark**. To pierwsze jest systemem wymiany danych, który umożliwia przesyłanie danych między różnymi etapami potoku przetwarzania. Apache Spark, natomiast, umożliwia przetwarzanie danych strumieniowych i transformację danych surowych w formę, która jest już gotowa do analizy.

W projekcie nie wykorzystano Hadoopa, co jest zgodne z przyjętym zakresem projektu (był traktowany jako opcjonalne narzędzie). Zamiast tego dane są przetwarzane lokalnie, a wyniki są zapisywane w strukturze katalogów projektu.

Struktura danych w projekcie podzielona jest na trzy katalogi:

- **data/raw** - przechowuje dane surowe, m. in. raporty XLSX/PDF zaczerpnięte ze źródeł zewnętrznych
- **data/processed** - przechowuje dane oczyszczone oraz zagregowane, które wykorzystywane są już prosto przez analizę tras
- **data/results** - przechowuje wyniki analiz takie jak porównania tras

W obecnej wersji projektu, możliwe jest uruchomienie pełnej analizy trasy:

1. Pobranie danych źródłowych (np. raporty XLSX/PDF z zewnętrznych źródeł)
2. Przetworzenie danych (czyszczenie, transformacja, agregacja)
3. Wyznaczenie trasy na podstawie przetworzonych danych
4. Porównanie różnych wariantów trasy według różnych funkcji kosztu

2.2 Potok danych

Potok danych został przygotowany w takim sposób, aby połączyć różne źródła danych w jeden spójny model kosztu przejazdu.

Podstawą analizy jest graf drogowy Warszawy - stworzony na podstawie danych z **OpenStreetMap** z pomocą biblioteki **OSMnx**. Graf ten jest następnie wzbogacany o dane:

- **ZDM** - dane dotyczące natężenia ruchu oraz średnich prędkości pojazdów (dane te są jedynie punktowe, więc są wykorzystywane do korekty kosztu przejazdu tylko dla tych krawędzi, dla których są dostępne)
- **Dane pogodowe** - dane dotyczące warunków pogodowych, które mogą wpływać na czas przejazdu, np. deszcz czy mgła

Potok obejmuje następujące etapy:

- Pobór danych
- Czyszczenie i transformację danych
- Agregację danych do postaci użytecznej w analizie tras
- Zapis wyników pośrednich i końcowych

Szczegółowy opis poszczególnych etapów potoku przedstawiono w kolejnych podpunktach.

2.2.1 Źródła danych

Podstawowym źródłem danych jest **OpenStreetMap**, który dostarcza szczegółową mapę sieci drogowej Warszawy. Dane pogodowe oraz te z **ZDM** służą do modyfikacji kosztu przejazdu przez konkretne odcinki grafu.

OpenStreetMap - dane z OSM są pobierane z pomocą biblioteki **OSMnx** i na ich podstawie tworzony jest graf drogowy Warszawy dla ruchu samochodowego. Węzły grafu to punkty sieci drogowej, np. skrzyżowania. Krawędzie to odcinki dróg. Jak już wspomniano wcześniej, jest to główne źródło danych.

Następnym źródłem danych to dane **ZDM Warszawa ARP** - dane z Systemu Automatycznych Pomiarów Ruchu. Dane te podają informacje o natężeniu ruchu w wybranych punktach warszawskiej sieci drogowej - z dokładnością do godziny oraz podziałem na kierunki ruchu. W projekcie wykorzystywane są do wyznaczania współczynnika **traffic_factor**. Współczynnik ten zwiększa koszt przejazdu przez daną ulicę, jeśli ma zwiększone natężenie ruchu. Jest to zdecydowanie najlepsze i najważniejsze źródło informacji jeżeli chodzi o rzeczywisty ruch w Warszawie. Koniec końców należy jednak pamiętać, że pomiary te mają **charakter punktowy** i nie dotyczą każdej warszawskiej ulicy.

Trzecim źródłem są dane o prędkości pojazdów pochodzące z ZDM. Są to średnie wartości prędkości dla konkretnych punktów pomiarowych. Dane te są wykorzystywane do wyznaczania współczynnika **speed_factor**. Współczynnik ten, analogicznie do **traffic_factor**, wpływa na koszt przejazdu przez daną ulicę, jeśli średnia prędkość jest niższa niż ta bazowa z **OSMnx**. Jest to jednak bardziej warstwa pomocnicza - duża część ulic nie ma danych o prędkościach.

Kolejne źródło danych to dane pogodowe pobierane z **Open-Meteo**. Dane te są wykorzystywane do wyznaczania współczynnika **weather_factor**, który może wpływać na koszt przejazdu przez daną ulicę, jeżeli warunki pogodowe nie są korzystne, czyli jeżeli np. pada wtedy deszcz. Jeżeli dane nie są dostępne, dla współczynnika przyjmowana jest wartość domyślna czyli 1.0

2.2.2 Pobór danych - Kafka i skrypty ingestujące

2.2.3 Czyszczenie i transformacja danych - Apache Spark, OSMnx, Pandas

2.2.4 Zapis danych

2.2.5 Wykorzystanie danych w analizie tras

3 Instalacja, konfiguracja oraz wykonanie projektu

3.1 Instalacja i zbudowanie projektu

Aktualną wersję rozwiązania można znaleźć w otwartym repozytorium na GitHub, pod linkem <https://github.com/JacobPW23/OpenNavigation>.

Aby zapewnić uniwersalność rozwiązania i prostotę obsługi i udostępniania postanowiono skonteneryzować projekt za pomocą technologii Docker. W celu wykorzystania OpenNavigation należy sklonować repozytorium, zbudować obraz oraz uruchomić kontener na podstawie zbudowanego obrazu:

```
> git clone https://github.com/JacobPW23/OpenNavigation
> cd OpenNavigation
> docker compose build
> docker compose up -d
```

Należy zwrócić uwagę, iż z powodu instalowania narzędzi Apache, zbudowanie obrazu Docker'a może chwilę zająć.

3.2 Konfiguracja skryptów analizy

Po udanym postawieniu kontenera, i sprawdzeniu jego działania za pomocą `docker compose ps`, dane zostaną pobrane automatycznie według potoku danych.

Skrypty wykonujący analizę najoptymalniejszej trasy na podstawie danych znajduje się pod następującymi ścieżkami:

- `/app/src/analysis/route_initial_analysis.py`
- `/app/src/analysis/osm_initial_analysis.py`

W tym raporcie opisane zostanie wykorzystanie pierwszego skryptu. Pozostałe zostaną opisane w ramach raportu końcowego.

Kluczowym punktem dla analizy jest zdefiniowana w tym pliku stała **ROUTES**, w której określone są punkty początkowe i końcowe, dla których ma zostać znaleziona optymalna trasa. Poniżej zamieszczono przykładową konfigurację zmiennej, domyślnie znajdującą się w skrypcie w języku Python:

```
ROUTES = [
    {
        "route_name": "Mokotowska -> Dobra",
        "origin": (52.218662, 21.017080),
        "destination": (52.244974, 21.020670),
    },
]
```

gdzie współrzędne *origin* oraz *destination* są podawane jako (*latitude*, *longitude*).

Aby uruchomić analizę tras, należy wykonać plik analizy tras po wprowadzeniu swojej konfiguracji. Komenda:

```
> docker compose exec opennavigation python \
    src/analysis/route_initial_analysis.py
```

3.3 Dane wyjściowe

Wyniki wykonania skryptu znajdują się w katalogu `/app/data/results/route_initial_analysis/`. Dla analizy danych istotne są 3 pliki wynikowe:

1. `summary.md` - podsumowanie wyniku analizy skryptu;
2. `routes_comparison.csv` - plik csv podsumowujący najważniejsze aspekty determinujące proponowane trasy: `distance_km`, `base_time_min`, `adjusted_time_min`, `traffic_impacted_edges`, `speed_impacted_edges`, `zdm_apr_edges`, `zdm_speed_edges`, `osmnx_only_edges`;
3. `route_edges.csv` - szczegóły krawędzi najbardziej optymalnej trasy znalezionej przez algorytm, takie jak `traffic_factor`, `speed_factor`, `traffic_speed_factor`, `measured_avg_speed_kmh`, `cost_source`.

4 Wstępna analiza problemu

4.1 Wnioski

5 Dalsze kierunki badania

6 Podsumowanie